

Guia 3 Spring-boot : SPRINT BOOT PAQUETES: DOMAIN + @DATA LOMBOK.

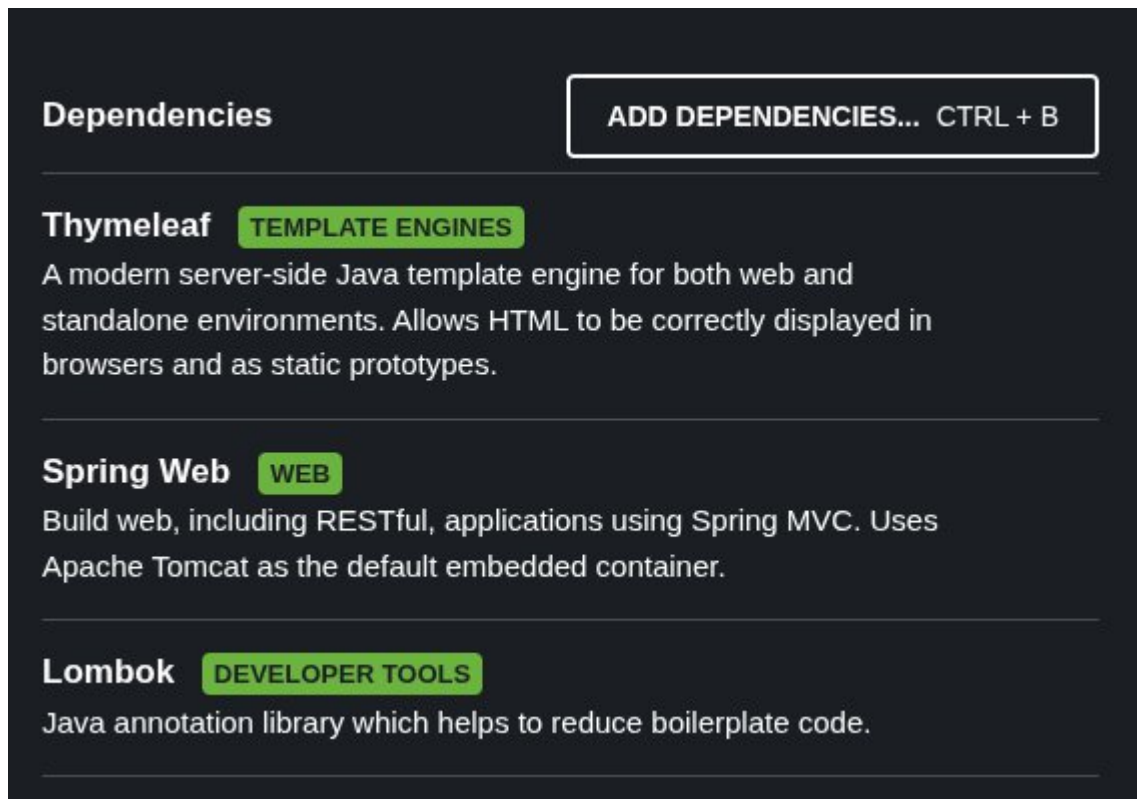
Nombre : Miguel Abraham Lacruz Medina.

Ficha : 3114733.

Programa : Analisis y desarrollo de software.

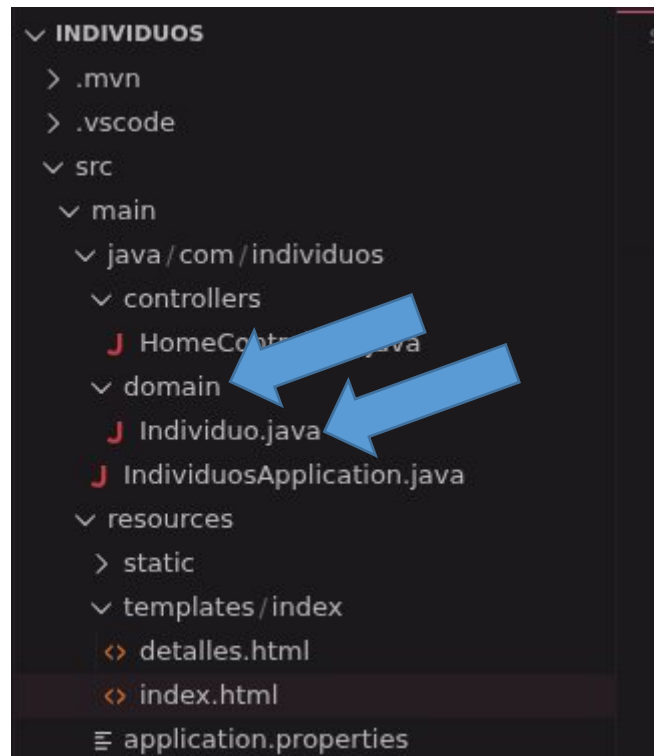
Profesora : Diana Delgado

- Creamos un proyecto nuevo y en la pagina de start.spring.io y agregamos las siguientes dependencias



- La dependencia **Thymeleaf** esta es para el servidor gráfico, es decir el que carga el contenido del **html**, el **Spring Web**, es quien monta el servidor web, **Lombok** es para nuestro ejemplo es quien facilita la creación de modelos.

Creamos la carpeta **domain** dentro de nuevo proyecto y dentro de esa carpeta creamos el **individuo.java**

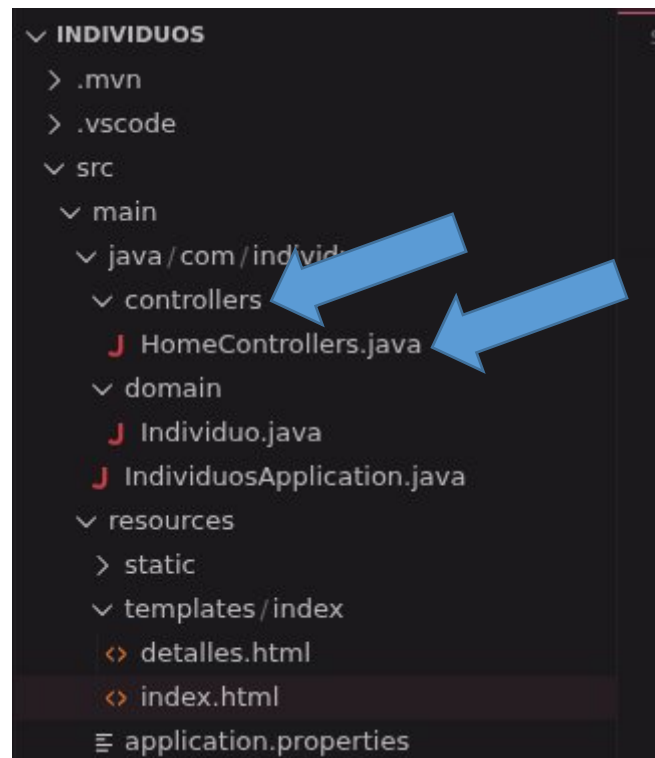


- Y le agregamos el siguiente contenido.

```
src > main > java > com > individuos > domain > J Individuo.java > ...
1  package com.individuos.domain;
2
3  import lombok.Data;
4
5  @Data
6  public class Individuo {
7      private String nombre;
8      private String apellido;
9      private int edad;
10     private String correo;
11 }
12
```

- Gracias a la dependencia **lombok** podemos usar el **@Data**, esto hace que lo que este dentro de la función publica **Individuo** se creen automáticamente los **Getters** y **Setters** para no tener que hacerlos manualmente, haciendo que el trabajo se entregue mas rapido. En este caso estamos agregando 4 valores : **nombre**, **apellido**, **edad** y **correo**.

Creamos la carpeta **controllers** dentro de nuestro proyecto y dentro de esa carpeta creamos nuestro controlador **HomeController.java**



- Y le agregamos la siguiente estructura

```
package com.individuos.controllers;

import java.util.ArrayList;
import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;

import com.individuos.domain.Individuo;
import org.springframework.ui.Model;

@Controller
public class HomeControllers {
    @GetMapping("/individuos")
    public String Home(Model modelo, @RequestParam(required = false) String q) {
        List<Individuo> lista = new ArrayList<>();

        // Creamos los datos
        Individuo ind1 = new Individuo();
        ind1.setNombre(nombre: "Miguel");
        ind1.setApellido(apellido: "Lacruz");
        ind1.setCorreo(correo: "Correo@gmail.com");
        ind1.setEdad(edad: 20);

        Individuo ind2 = new Individuo();
        ind2.setNombre(nombre: "Andres");
        ind2.setApellido(apellido: "Vega");
        ind2.setCorreo(correo: "Andres@gmail.com");
        ind2.setEdad(edad: 21);

        Individuo ind3 = new Individuo();
        ind3.setNombre(nombre: "Jose");
        ind3.setApellido(apellido: "Arteaga");
        ind3.setCorreo(correo: "Jose123@gmail.com");
        ind3.setEdad(edad: 27);
    }
}
```

```

        lista.add(ind1);
        lista.add(ind2);
        lista.add(ind3);

        // Lógica de filtrado
        List<Individuo> listaFiltrada = lista;

        if (q != null && !q.isEmpty()) {
            listaFiltrada = lista.stream()
                .filter(i -> i.getNombre().toLowerCase().contains(q.toLowerCase()))
                .toList();
        }

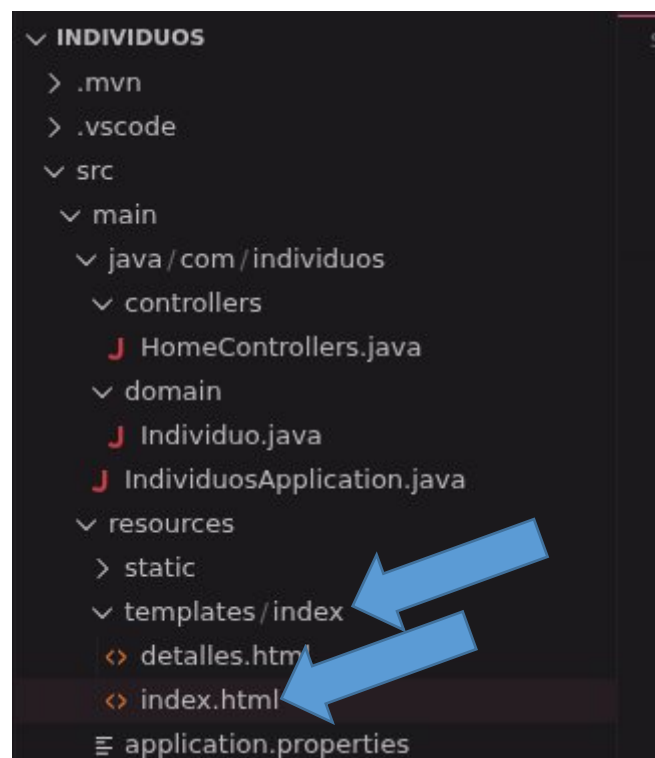
        modelo.addAttribute(attributeName: "lista", listaFiltrada);
        // Opcional: para que el texto se quede en el input
        modelo.addAttribute(attributeName: "q", q);
        return "index/index";
    }

    @GetMapping("/detalles")
    public String Detalles( @RequestParam String nombre,
                           @RequestParam String apellido,
                           @RequestParam int edad,
                           @RequestParam String correo,
                           Model modelo
                           ){
        Individuo individuo = new Individuo();
        individuo.setNombre(nombre);
        individuo.setApellido(apellido);
        individuo.setEdad(edad);
        individuo.setCorreo(correo);
        modelo.addAttribute(attributeName: "ind", individuo);
        return "index/detalles";
    }
}

```

- Lo que hace el código reciente mostrado es primero tener la ruta de **“/individuos”** que es la ruta donde veremos a los usuarios que tenemos en la lista y se crean 3 individuos partiendo del objeto que acabamos de crear **individuo.java**, los cuales tienen automáticamente al inicio un **Set** el cual este es para agregarles un valor, y en este ejemplo se creo **3 objetos** los cuales tienen un **nombre, apellido, correo y teléfono**.
- Y en ese mismo bloque como es el encargado de la ruta **“/individuos”** se agrega una segunda lista llamada **ListaFiltrada** que es la que se encarga de Filtrar por **nombre**. Y este mandara todo al modelo **modelo** y retornara todo a la vista **“index/index”**.
- En el segundo bloque tenemos la ruta **“/detalles”** la cual esta ruta se encarga de la vista detalles cuando demos clic a **ver** muestre los datos del que acabamos de dar clic, este mostrada **nombre, apellido, edad y correo** y todo eso se envía al modelo **modelo** y retornara a la vista **“index/detalles”**.

Ahora Creamos la carpeta **index** dentro de **templates** y dentro de esa carpeta creamos la vista **index.html** la cual mostrada el contenido de la lista que hemos creado



- Y le agregamos el siguiente contenido

```

<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org" lang="es">
<head>
  <meta charset="UTF-8">
  <title>Index</title>
</head>
<body>
  <h1>Individuos</h1>

  <form th:action="@{/individuos}" method="get">
    <input type="text" name="q" th:value="${q}" placeholder="Buscar persona">
    <button type="submit">Buscar</button>
  </form>

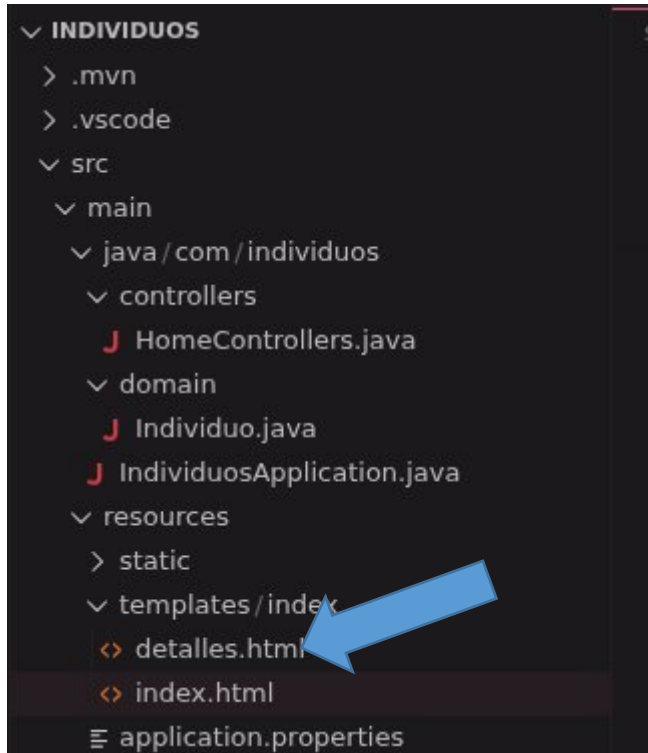
  <br>

  <table border="1">
    <thead>
      <tr>
        <th>Nombre</th>
        <th>Apellido</th>
        <th>Edad</th>
        <th>Correo</th>
        <th>Acciones</th>
      </tr>
    </thead>
    <tbody>
      <tr th:each="l : ${lista}">
        <td th:text="${l.nombre}"></td>
        <td th:text="${l.apellido}"></td>
        <td th:text="${l.edad}"></td>
        <td th:text="${l.correo}"></td>
        <td>
          <a th:href="@{/detalles(nombre=${l.nombre}, apellido=${l.apellido}, edad=${l.edad}, correo=${l.correo})}">
            Ver
          </a>
        </td>
      </tr>
    </tbody>
  </table>
</body>
</html>

```

- Lo que hace este código es agregar un **input** del tipo text (este es para que nos permita escribir el nombre del que deseamos buscar), un **botón** (este nos permitirá filtrar lo que hemos escrito por el input), y una tabla que contiene los nombres, apellidos, correos y teléfonos de los recién agregados del controlador **HomeControllers.java**, dentro de la tabla este recorrerá los valores mandados de la lista desde el **HomeControllers.java** para poder imprimir su contenido en pantalla.

Ahora creamos la vista **detalles.html** dentro de la carpeta **index**



- Con el siguiente contenido

```

<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>XD</title>
</head>
<body>
  <h1>Detalles</h1>
  <table>
    <thead>
      <tr>
        <td>Nombre</td>
        <td>APellido</td>
        <td>Edad</td>
        <td>Correo</td>
      </tr>
    </thead>
    <tbody>
      <tr th:each="i : ${ind}">
        <td th:text="${i.nombre}"></td>
        <td th:text="${i.apellido}"></td>
        <td th:text="${i.edad}"></td>
        <td th:text="${i.correo}"></td>
      </tr>
      <a th:href="@{/individuos}">Regresar</a>
    </tbody>
  </table>
</body>
</html>

```

- Esta vista muestra el contenido del que le hemos hecho clic en el contenido de la etiqueta `<a>` que se encuentra en la vista `index.html`, mostrando el valor de cada uno de las personas tales como el : **nombre**, **apellido**, **correo** y **telefono**.

Entramos a la ruta “/individuos”

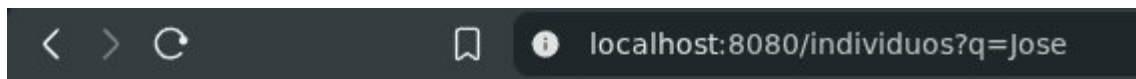


Individuos

Nombre	Apellido	Edad	Correo	Acciones
Miguel	Lacruz	20	Correo@gmail.com	Ver
Andres	Vega	21	Andres@gmail.com	Ver
Jose	Arteaga	27	Jose123@gmail.com	Ver

- Como vemos los datos del controlador **HomeController.java** fueron enviados correctamente mostrarlo los valores en pantalla gracias a la vista y el recorriendo que hace.

Ahora filtraremos por algun nombre

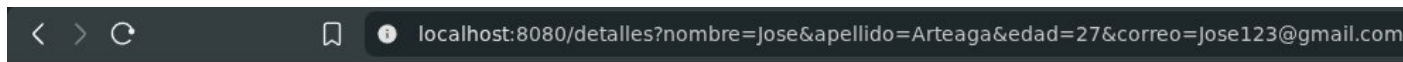


Individuos

Nombre	Apellido	Edad	Correo	Acciones
Jose	Arteaga	27	Jose123@gmail.com	Ver

- Al colocar el nombre y darle al boton **Buscar** nos permite filtrar por el nombre que hemos colocado

Ahora precionamos en la parte **Acciones** el boton que dice **Ver**



Detalles

[Regresar](#)

Nombre APellido Edad Correo

Jose Arteaga 27 Jose123@gmail.com

- Mostrando que funciona correctamente y nos permite ver los valores de esa persona como se ve en el ejemplo.